Teach Module 43: Bitbucket Pipelines and CI/CD Integration

Module Topic

Module 43 focuses on **Bitbucket Pipelines and CI/CD Integration**.

The purpose of this module is to understand how to automate build, test, scan, and deployment workflows using Bitbucket Pipelines.

The course document was used only to identify the module topic. The teaching content below is written independently.

What Is Bitbucket Pipelines?

Bitbucket Pipelines is Bitbucket Cloud's built-in CI/CD service.

It lets you define automated workflows in a file named:

```
bitbucket-pipelines.yml
```

This file lives at the root of your repository.

Whenever code is pushed, a pull request is updated, or a branch rule is matched, Bitbucket can run the pipeline automatically.

At a high level:

```
Developer pushes code
        ↓
Bitbucket detects the change
        ↓
Pipeline starts
        ↓
Build → Test → Scan → Package → Deploy
        ↓
Pull request or deployment is marked pass/fail
```

Each pipeline step usually runs inside a Docker container. That gives the pipeline a clean, predictable environment for every run.

Why CI/CD Matters

CI/CD helps teams avoid relying on manual checks.

Instead of asking developers to remember every build, test, and deployment command, the repository itself defines the workflow.

This gives the team:

- Faster feedback

- Repeatable builds
- Fewer manual mistakes
- Safer pull requests
- More consistent deployments
- Better visibility into failures

For a Java team, CI/CD commonly includes:

- Compiling the code
- Running unit tests
- Running integration tests
- Running static analysis
- Scanning dependencies for vulnerabilities
- Packaging a `.jar`
- Publishing build artifacts
- Deploying to staging or production

Core Vocabulary

Pipeline

A **pipeline** is the full workflow that runs in response to a trigger.

Example:

```
Build → Test → Scan → Deploy
```

Step

A **step** is one unit of work inside a pipeline.

Examples:

```
Compile application
Run unit tests
Scan dependencies
Deploy to staging
```

Script

A **script** is the list of commands a step runs.

Example:

```yaml
script:
  - mvn clean test
```

Artifact

An **artifact** is a file produced by one step and passed to later steps.

For a Java project, a common artifact is:

```
target/*.jar
```

Cache

A **cache** stores dependencies between pipeline runs.

For Maven projects, this can prevent downloading the same dependencies repeatedly.

Example:

```yaml
caches:
  - maven
```

Deployment Environment

A **deployment environment** is a named target such as:

```
test
staging
production
```

Manual Gate

A **manual gate** pauses the pipeline until someone approves the next step.

This is commonly used before staging or production deployment.

Basic Java Maven Pipeline

A simple Maven pipeline can look like this:

```yaml
image: maven:3.9-eclipse-temurin-21

pipelines:
  default:
    - step:
        name: Build and Test
        caches:
          - maven
        script:
          - mvn clean test
```

Explanation:

- `image` selects the Docker image used by the pipeline.
- `pipelines` defines the workflows.
- `default` runs for normal pushes unless a more specific rule matches.
- `step` defines one job.
- `script` lists the commands to execute.
- `mvn clean test` cleans the project and runs tests.

Separating Build and Test Steps

In real projects, teams often separate build and test work.

```
image: maven:3.9-eclipse-temurin-21

pipelines:
  default:
    - step:
        name: Build Application
        caches:
          - maven
        script:
          - mvn clean package -DskipTests
        artifacts:
          - target/*.jar

    - step:
        name: Run Tests
        caches:
          - maven
        script:
          - mvn test
```

The first step builds the application and stores the `.jar` as an artifact.

The second step runs tests.

Pull Request Validation

Pull request validation protects shared branches.

Example:

```
image: maven:3.9-eclipse-temurin-21

pipelines:
  pull-requests:
    "**":
      - step:
          name: Pull Request Validation
          caches:
            - maven
          script:
            - mvn clean verify
```

This means:

```
For every pull request, run Maven verification.
If verification fails, the PR should not be merged.
```

Parallel Execution

Some pipeline checks can run at the same time.

For example:

```
image: maven:3.9-eclipse-temurin-21

pipelines:
  default:
    - step:
        name: Build Application
        caches:
          - maven
        script:
          - mvn clean package -DskipTests
        artifacts:
          - target/*.jar

    - parallel:
        - step:
            name: Unit Tests
            caches:
              - maven
            script:
              - mvn test

        - step:
            name: Static Analysis
            caches:
              - maven
            script:
              - mvn checkstyle:check
```

Parallel execution gives developers faster feedback because independent checks run concurrently.

If Checkstyle is not configured in the project, the static analysis command can be replaced with:

```
mvn verify
```

Quality Gates

A **quality gate** is a rule that must pass before code is merged or deployed.

Examples:

- Build must succeed
- All tests must pass
- Code coverage must meet the threshold
- No critical vulnerabilities are allowed
- Pull request must have approval

- Required merge checks must pass

Quality gates help prevent unsafe or broken code from reaching important branches or environments.

Code Insights

Code Insights can show useful reports in Bitbucket pull requests.

Examples include:

- Static analysis results
- Security scan results
- Unit test reports
- Code coverage
- Artifact links
- Build status

This makes review easier because developers can see quality feedback directly in the pull request.

Bitbucket Pipes

Bitbucket Pipes are reusable pipeline integrations.

Instead of writing a long deployment or notification script yourself, you can call a pipe.

Conceptual example:

```
script:
  - pipe: atlassian/example-pipe:1.0.0
    variables:
      SOME_VALUE: "example"
```

Pipes are often used for:

- Cloud deployments
- Slack notifications
- Docker image publishing
- Security scans
- Artifact publishing

Repository Variables and Secrets

Secrets should not be hardcoded in `bitbucket-pipelines.yml`.

Bad example:

```
script:
  - aws configure set aws_secret_access_key "my-secret-key"
```

Better example:

```
script:
```

```
  - aws configure set aws_secret_access_key "$AWS_SECRET_ACCESS_KEY"
```

The value should be stored in Bitbucket as a repository, workspace, or deployment variable.

Common variables:

```
APP_ENV
DEPLOY_TARGET
AWS_REGION
DOCKER_USERNAME
DOCKER_PASSWORD
```

OIDC for Secure Deployments

OIDC stands for OpenID Connect.

In CI/CD, OIDC can allow Bitbucket Pipelines to request short-lived cloud credentials from a cloud provider.

This is safer than storing long-lived cloud access keys in repository variables.

Example shape:

```yaml
pipelines:
  default:
    - step:
        name: Deploy with OIDC
        oidc: true
        script:
          - echo "Request short-lived credentials from the cloud provider"
```

The exact commands depend on the cloud platform, such as AWS, Azure, or Google Cloud.

Manual Deployment Gate

A deployment step can be configured so it only runs after manual approval.

```yaml
image: maven:3.9-eclipse-temurin-21

pipelines:
  branches:
    main:
      - step:
          name: Build Release Artifact
          caches:
            - maven
          script:
            - mvn clean package
          artifacts:
            - target/*.jar

      - step:
```

```
          name: Deploy to Staging
          deployment: staging
          trigger: manual
          script:
            - echo "Deploying JAR to staging..."
            - ls target
```

This means:

```
When code reaches main, build the release artifact.
Then pause before deploying to staging.
A human must approve the deployment step.
```

Recommended Practice Exercises

Exercise 1: Create a Basic Java Pipeline

Create a `bitbucket-pipelines.yml` file for a Maven Java project.

Goal:

```
On every push:
1. Run mvn clean compile
2. Run mvn test
```

Practice focus:

- Pipeline structure
- Docker image selection
- Basic CI automation

Exercise 2: Separate Build and Test Steps

Split the pipeline into two steps:

```
Step 1: Build the application
Step 2: Run tests
```

Add an artifact from the build step.

Practice focus:

- Multi-step pipelines
- Artifacts
- Step isolation

Exercise 3: Add Pull Request Validation

Create a pipeline that runs when a pull request is opened or updated.

Goal:

```
For every PR:
- Compile the project
- Run tests
- Fail the PR if tests fail
```

Practice focus:

- Pull request pipelines
- Merge safety
- Automated quality checks

Exercise 4: Run Tests in Parallel

Create parallel steps for:

```
Unit tests
Integration tests
Static analysis
```

Practice focus:

- Parallel execution
- Faster feedback
- Independent pipeline jobs

Exercise 5: Add a Security Scan

Add a dependency vulnerability scan.

Goal:

```
Pipeline fails if serious dependency vulnerabilities are found.
```

Practice focus:

- Security in CI
- Dependency scanning
- Quality gates

Exercise 6: Use Repository Variables

Create repository variables such as:

```
APP_ENV
DEPLOY_TARGET
API_BASE_URL
```

Then reference them inside the pipeline.

Practice focus:

- Environment variables
- Configuration outside code

- Avoiding hardcoded values

Exercise 7: Add a Manual Deployment Gate

Create a deployment step that does not run automatically.

Goal:

```
Build and test automatically.
Deploy to staging only after manual approval.
```

Practice focus:

- Manual gates
- Deployment control
- Staging environments

Exercise 8: Create a Staging and Production Flow

Build this workflow:

```
Build → Test → Deploy to Staging → Manual Approval → Deploy to Production
```

Practice focus:

- Deployment environments
- Artifact promotion
- Controlled production releases

Exercise 9: Add Code Insights or Test Reports

Configure the build so test results are visible in Bitbucket.

Goal:

```
Generate JUnit-style test reports from Maven Surefire.
Use the pipeline results to inspect test failures.
```

Practice focus:

- Test visibility
- Pull request feedback
- Debugging failed builds

Exercise 10: Use a Bitbucket Pipe

Use a prebuilt Bitbucket Pipe for a common task.

Examples:

- Send a Slack notification
- Deploy to AWS
- Upload an artifact
- Run a scanner

Practice focus:

- Reusable integrations
- Pipeline simplification
- Third-party services

Exercise 11: Add Branch-Specific Pipelines

Create different behavior for different branches:

```
feature/* branches: run tests only
develop branch: build and test
main branch: build, test, and prepare production deployment
```

Practice focus:

- Branch-based automation
- Release workflows
- Environment separation

Exercise 12: Secure Deployment with OIDC

Configure a pipeline step with:

```
oidc: true
```

Then use it to request temporary cloud credentials.

Practice focus:

- Cloud deployment security
- Short-lived credentials
- Avoiding stored access keys

Lab: Build a Bitbucket CI/CD Pipeline for a Java App

Goal

Create a working `bitbucket-pipelines.yml` that builds, tests, scans, stores an artifact, and prepares a manual deployment step.

Prerequisites

You need:

- A Bitbucket repository
- A Java Maven project
- A `pom.xml` file
- Bitbucket Pipelines enabled

If you do not have a project, use any simple Spring Boot or Maven Java app.

Part 1: Create the Pipeline File

In the root of your repo, create:

```
bitbucket-pipelines.yml
```

Add this:

```yaml
image: maven:3.9-eclipse-temurin-21

pipelines:
  default:
    - step:
        name: Build and Test
        caches:
          - maven
        script:
          - mvn clean test
```

Commit and push:

```
git add bitbucket-pipelines.yml
git commit -m "Add basic Bitbucket pipeline"
git push
```

Check Bitbucket > Pipelines and confirm the pipeline runs.

Part 2: Separate Build and Test

Replace the file with:

```yaml
image: maven:3.9-eclipse-temurin-21

pipelines:
  default:
    - step:
        name: Build Application
        caches:
          - maven
        script:
          - mvn clean package -DskipTests
        artifacts:
          - target/*.jar

    - step:
        name: Run Tests
        caches:
          - maven
        script:
          - mvn test
```

Push again and verify both steps run.

Part 3: Add Parallel Quality Checks

Update the file:

```
image: maven:3.9-eclipse-temurin-21

pipelines:
  default:
    - step:
        name: Build Application
        caches:
          - maven
        script:
          - mvn clean package -DskipTests
        artifacts:
          - target/*.jar

    - parallel:
        - step:
            name: Unit Tests
            caches:
              - maven
            script:
              - mvn test

        - step:
            name: Static Analysis
            caches:
              - maven
            script:
              - mvn checkstyle:check
```

If your project does not have Checkstyle configured, replace that command with:

```
mvn verify
```

Part 4: Add Pull Request Validation

Add this section:

```
  pull-requests:
    "**":
      - step:
          name: Pull Request Validation
          caches:
            - maven
          script:
            - mvn clean verify
```

Now your pipeline validates pull requests before merge.

Part 5: Add Manual Staging Deployment

Add a branch pipeline for `main`:

```
branches:
  main:
    - step:
        name: Build Release Artifact
        caches:
          - maven
        script:
          - mvn clean package
        artifacts:
          - target/*.jar

    - step:
        name: Deploy to Staging
        deployment: staging
        trigger: manual
        script:
          - echo "Deploying JAR to staging..."
          - ls target
```

This creates a manual gate before staging deployment.

Part 6: Use Repository Variables

In Bitbucket, go to:

```
Repository settings → Pipelines → Repository variables
```

Create:

```
APP_ENV=staging
DEPLOY_TARGET=my-staging-server
```

Update the deployment step:

```
script:
  - echo "Deploying to $APP_ENV"
  - echo "Target server is $DEPLOY_TARGET"
  - ls target
```

Final Expected Pipeline Capabilities

Your final pipeline should include:

- Default pipeline
- Pull request pipeline
- Main branch deployment pipeline

- Build artifact
- Parallel checks
- Manual staging deployment
- Repository variables

Lab Challenge

Modify the pipeline so that:

```
feature/* branches run only tests
develop branch builds and tests
main branch builds, tests, and allows manual deployment
```

Deliverables

Submit:

1. Your `bitbucket-pipelines.yml` file
2. Screenshot of a successful pipeline run
3. Screenshot of the manual deployment step
4. Short explanation of what each step does

Quick Review Questions

1. What file defines Bitbucket Pipelines?
2. What is the difference between a pipeline and a step?
3. Why should secrets not be hardcoded in the YAML file?
4. What is the purpose of a manual deployment gate?
5. Why might teams run tests and scans in parallel?
6. What is an artifact?
7. How does pull request validation protect a shared branch?
8. Why is OIDC safer than storing long-lived cloud credentials?

Summary

Bitbucket Pipelines allows Java teams to automate the path from code change to validated build.

A strong pipeline should:

- Build the application
- Run tests automatically
- Surface failures early
- Store useful artifacts
- Protect pull requests
- Use variables and secrets safely
- Support controlled deployments
- Prefer short-lived credentials where possible

The main idea is simple:

```
The repository should know how to prove that the code is ready.
```